

Energy efficient configuration unification and compression for CGRAs

Mohammad Hossein Sargolzaei^a, Siamak Mohammadi^{b,c}

^a School of Electrical and Computer Engineering, University of Tehran, Tehran, Iran

^b Dependable System Design Laboratory, School of ECE, University of Tehran, Tehran, Iran

^c School of Computer Science, Institute of Fundamental Sciences (IPM), Tehran, Iran

ARTICLE INFO

Keywords:

CGRA
Energy efficiency
Context-switching power reduction
Instruction-set CGRA simulator

ABSTRACT

Energy efficiency has become one of the most important factors in today's processing platforms. Coarse-grained reconfigurable architectures (CGRAs) have turned out to perform well in this area. In this paper, we have proposed two methods to enhance their energy efficiency. The first method decreases the power consumption of a CGRA by reducing the number of occurring transitions of the context switching process. The second one decreases the energy consumption of a CGRA through volume shrinkage of its context memory. Our results show up to 83% of the context memory energy can be reduced by using our proposed method. In addition, a high level instruction-set simulator for CGRA has been presented in this paper.

1. Introduction

High performance and energy efficiency are two desired factors of every computing platform, especially in embedded systems world. Due to growing variety and complexity of embedded applications, the importance of these two factors has increased. However, having both of them in one processing platform is so rare. Coarse-grained reconfigurable array (CGRA) architectures are one of these rare and successful platforms. Nowadays, CGRAs are used in many process intensive domains, such as multimedia [1–3], healthcare systems [4], digital signal processing [5–8], and so on.

Efficiency of a CGRA comes from its unique program execution model which is based on dynamic datapath reconfiguration in runtime. A CGRA is a VLIW processor with a 2D reconfigurable execution stage. In a VLIW processor, a set of instructions will be issued in one clock cycle, but it can take more time for execution. In CGRAs, each package of instruction, which is called a “context” is a 2D datapath. A CGRA context can be a simple datapath with a couple of clock cycles delay, or a highly pipelined one with more than hundreds clock cycles delay. Fetching a 2D datapath instead of a single instruction has many advantages, such as removing unnecessary temporal variables, reducing number of accesses to register file, and so on. CGRA could save plenty of energy consumption by removing these extra steps from the program execution model. Accordingly, high energy efficiency of CGRAs is due to their program execution model.

Aside from advantages of the unique and powerful program execution model, CGRAs have some dark sides too. The main part of the model is fetching a new context, which is called context switching (CS)

process. In the CS process, a new configuration has to be loaded from the context memory (CM) to each processing element (PE). Based on the number of PEs of the CGRA, the number of configuration words of each context can be too large. On the other hand, due to growing complexity of CGRA applications, the number of needed contexts per application is increasing, and consequently, the cost of the CS process is rising dramatically.

To show the cost of the CS process, an example is shown here. Suppose a simple sequential 4-tap FIR filter program which is compiled to run on a simple 4×4 CGRA. After compiling and mapping steps of the program, 4 different contexts will be generated which need 2Kb CM to be saved. However, if that program goes to a simple in-order RISC processor, it will only need 0.5Kb. The required higher number of configuration bits compared to RISC processor makes two different problems in terms of energy efficiency in CGRAs. First, these large configuration bits have to be stored in a big context memory inside the CGRA, where it consumes high energy. On the other hand, in the CS process of CGRA program execution model, more than 128K configuration bits have to be fetched from the CM; whereas, for a RISC processor, it is only about 2.5K bits. The high number of fetched configuration bits causes an enormous number of transitions in the CGRA modules. Hence, energy efficiency of the CGRA will furthermore decrease because of these transition bits in addition to the big CM module.

To tackle this problem efficiently, the CS process has to be well understood. Before executing a program on a CGRA, it has to be transformed into several contexts. Based on the model of the CGRA program execution, at any instance of time, only one context is placed in the PE array and others have to be on standby in the CM. In the CS

E-mail addresses: mh.sargolzaei@ut.ac.ir (M.H. Sargolzaei), smohamadi@ut.ac.ir (S. Mohammadi).

<https://doi.org/10.1016/j.micpro.2018.06.010>

Received 31 December 2017; Received in revised form 15 April 2018; Accepted 14 June 2018

Available online 03 July 2018

0141-9331/ © 2018 Elsevier B.V. All rights reserved.

process, a context will be fetched from the CM and stored in the configuration register of each PE. Inside of each context, the address of the next context is defined, and when its conditions are satisfied, the CS process will be started. During the CS process, the data memory (DM) goes to SLEEP mode, the CM goes to Config.READ mode and the PE array goes to Config.Write mode. The CS process has three different phases: SLEEP, WRITE, and AWAKE. First, the control unit sends a request for context switching and takes all PEs and DM to SLEEP mode. Then, it sends the address of the next context to the CM for fetching a new context and takes PEs to the Config.Write mode. Its delay can be one or more clock cycles. Finally, the DM and all PEs go to the working mode.

Along with the CS process, new configuration bits have to be transmitted from the CM to all PEs by CM channels. CM channels are global busses connecting the context memory to all PEs. By shrinking transistors' sizes, the effect of these global wires becomes dominant. In the CS process, configuration bits have to be shifted into these channels and drive their large capacitances.

In the last decade, many different methods have been proposed to decrease the amount of energy consumption of the CS process in CGRAs. Main trends to solve this problem can be divided into five categories: context compression, memory optimization, mapping technique, architecture changing, and VLSI techniques. Some of important techniques of these categories are described in Section 3.

In this paper, we are proposing multiple techniques to reduce energy consumption of CGRAs by decreasing energy of the CS process. Our contributions are:

- Reducing the number of transition bits of the CS process, and respectively dynamic energy consumption of the CGRA, by unifying configuration words.
- Presenting a configuration compression method to reduce the CM module size as well as its static energy consumption.
- Describing some techniques to improve efficiency of the proposed methods.
- Categorizing different CM networks into three main groups and investigating the effect of our proposed methods on these groups.
- Presenting a high level instruction-set simulator for our CGRA, which can predict delay and energy of the CGRA by running applications.

This paper is organized as follows. Section 2 describes some basic information about components of a CGRA and its working phases. In addition, properties of the selected benchmarks are presented in the same section. Some previous works to decrease power or energy consumption of the CGRAs are presented in Section 3. The configuration unification method is described in Section 4. In Section 5, the proposed configuration compression method and its results are shown. Techniques to improve efficiency of our proposed methods are presented in Section 6. Section 7 presents our high level instruction set simulator. Section 8 describes our experimental setup and compares our results with previous works. Finally, a conclusion of our proposed methods are shown in Section 9.

2. Preliminaries

CGRAs are one of the application specific architectures designed for energy efficiency. When a CGRA is designed, its ISA and architecture have to be optimized for the target group of applications. Hence, many CGRA architectures are proposed for different applications with their own ISA. On the other hand, there are many application types in the embedded system world with different properties. Therefore, categorizing these architectures in a few groups is impossible. In this section, some basic information about the basic CGRA architecture is presented. In addition, some properties of our selected applications (benchmarks) for this paper will be discussed.

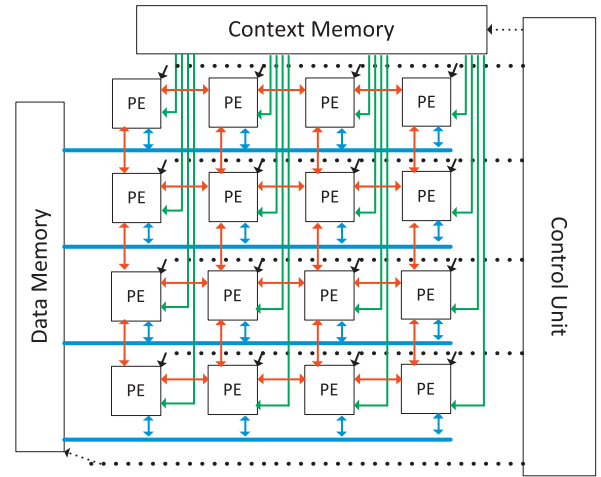


Fig. 1. A basic CGRA architecture.

2.1. CGRA architecture

The architecture of a CGRA is designed based on its target application group, although some components and structures are common in all CGRAs. In this paper, a basic CGRA with only the common features is designed and implemented. In a simple CGRA, there are two different memory units, one is for storing contexts (CM) and another is for data (DM). In addition, there is one global control unit and an array of processing elements (PEs). Mainly in different CGRAs, these modules are identical and only the ISA of PEs will be different and/or a small change in the program execution model of the CGRA can be seen. Fig. 1 shows a CGRA with a basic architecture.

There are four different networks in Fig. 1. One network is responsible for transmitting data between DM and PEs (blue horizontal lines). Another network transmits configurations from the CM to PEs (green vertical lines). Control signals go through the control network (black dotted lines); and finally, PE-to-PE data network (local red lines) is responsible for creating the reconfigurable datapath of the application.

The control network in all CGRAs is a point-to-point network; although there could be different topologies and architectures for other networks of CGRAs. In many CGRAs, there is a central DM module, which has a single read/write port and a shared bus for each row (column) of the PE array [9,10]. As for the PE-to-PE network, CGRAs mostly use a mesh topology [1,9–13], because of its flexibility to construct different datapaths. In this paper, our focus is on the CM network, used in the CS process. The CM module could have a single read port for each PE, each row (column), or all PEs. In the first option, the CM has one single port for each PE [14–17]. In these CGRAs, the CS process is done in only one clock cycle, while the wiring cost of this architecture is high. Hence, in some recent CGRAs [12,18], designers have used this architecture inside of PE clusters.

In the second CM network architecture, the CM module has one read port for each row (column) of the PE array [2,19,14]. In these CGRAs, the CM module shifts configuration words of all PEs of that row (column) in that port sequentially. In this architecture, the CS process takes 'N' clock cycles ('N' is the number of PEs in each row (column) of PE array). Because of shifting configuration bits of different PEs in one shared bus, the number of occurred transitions in the CS process is more than the previous architecture, but its wiring cost is less and also its implementation is simpler.

In the third CM network, there is only one read port for the entire PE array. In this architecture, CGRA has a big shared bus from the CM to all PEs. In the CS process, all configuration words will be shifted into that bus. The CS process delay in this architecture is too high ($N \times N$). To decrease delay and energy of the CS process in this model, usually more

than one configuration registers are located in each PE [2,5,17,19,20].

There are different types of PEs in various CGRAs. In many CGRAs [11,12,14,17,19,21], PEs consist of a single ALU module with a small register file, which is responsible for executing only one instruction of the datapath. But in some CGRAs [1,20,22,23], the PE module is more complicated. For example, in EGRA (Expression-Grained Reconfigurable Array) [23], each PE is a complex multi-layer module.

In this paper, we intend to lower energy consumption of the CGRA through reducing the CS cost. Therefore, we have selected the common features of the CGRAs as our basic architecture. The properties of our basic CGRA are:

- PE array contains 16 PEs in a 4×4 mesh network.
- All PEs consist of a single cycle execution ALU.
- The DM module has one port for each row of PE array.
- Mesh topology is selected for PE-to-PE network.
- Token passing technique is used as control flow in the datapath of CGRA.

We have used token passing control flow because of its strong effect on the energy efficiency of CGRAs [24]. In addition to above parameters, to investigate the effect of our proposed methods on the different CM networks, all three different models of the CM network are implemented. Thus, we have three different CGRAs with the same ISA and architecture. The selected ISA of the basic CGRA is designed and optimized based on our benchmarks. In the selected ISA, each instruction is a 32-bit word. Each instruction contains six different fields: opcode (5 bits), two control flow fields (10 bits), two input fields (6 bits) and immediate number (11 bits).

2.2. The context switching process

When a program is to run on a CGRA, it should be compiled into multiple contexts. Each context contains a small piece of program and one or more addresses to the other context(s). In each context, the last executed instruction is always a request for context switching (CS). Each context may contain one branch instruction or multiple conditional branch instructions. Fig. 2 shows different contexts of a 10×10 matrix multiplication application.

As shown in Fig. 2, contexts #0 and #3 have one branch instructions but contexts #1 and #2 have more than one branch instructions. When a context has more than one branch instruction, the number (or

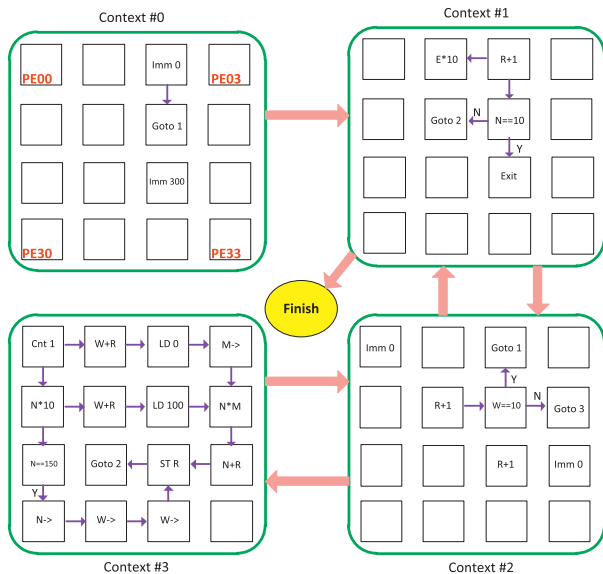


Fig. 2. Execution process of a program on a CGRA.

address) of the next context will be determined at the execution time, not at the compile time.

When a CS request goes to the control unit, it will deactivate all PEs and the DM, and the next context address will be sent to the CM module with a read command. This phase of CS always takes one clock cycle. Then, the CM module reads a new context from the incoming address and sends its configuration words to all PEs. In this phase, configuration words have to be written to the configuration register of each PE. The delay of this phase depends on the CM network model, as we mentioned before. At last, when the transmission of configuration words is finished, the control unit wakes up the DM module and PEs in one clock cycle.

Mainly, the CS process energy is related to three different parameters: number of requested context switching processes, model of the CM network, number of transition bits in the CM network. The number of requested context switching processes depends on the number of loops in the input program and it cannot be changed in run-time. However, at the architecture-level, the number of accesses to the CM module can be decreased by using more than one configuration register in each PE [21,25].

The CM network model has a strong impact on the CS process cost. In the first CM network, there is a global channel per PE to transfer the configuration word from the CM module to each PE. Hence, the CS process delay in this model is the least, whereas the amount of energy for each transmitted bit is high. In the second model, there is a shared bus for each row (or column) of the PE array. The delay and energy-per-transition-bit of this model are moderate. Finally, in the last one, there is a big shared bus with ' $N \times N$ ' clocks delay and big energy-per-transition-bit. Hence, choosing the perfect CM network model for the target applications must be energy aware for the CGRA.

On the other hand, the number of transition bits is directly proportional to the dynamic power of every circuit. When bits of successive shifted configuration words are not identical, some transition bits will be generated. In addition to these transition bits, some extra transition bits will be generated inside each PE at the configuration writing time.

2.3. Benchmark applications

CGRA is an application specific architecture, and properties of the target application group have a strong effect on the ISA and architecture of the CGRA. Therefore, we have chosen different domains of applications to investigate the effect of our methods in different categories. Table 1 shows the main properties of the selected applications and their categories.

As shown in Table 1, twenty different applications in four different groups are selected as the benchmark suite in this paper. Seven mathematical process intensive applications are selected from the kernels of the Livermore loops benchmark [26], designed to investigate the processing efficiency of parallel systems. These applications have small number of branches and memory bandwidth. In the second group, five well-known sorting applications [27] are selected to investigate the effect of proposed methods on the CGRAs, designed for memory processing. These applications have high memory usage and many conditional branches causing many CS requests. In the next group of applications, three image processing filters [28] are selected. These filters are implemented based on matrix processing which means high memory bandwidth and many internal loops. Finally, the last group of applications contains five common digital signal processing (DSP) kernels. The first two kernels are two common filters and others are three vector-based processing application [29]. These applications have small memory bandwidth and small number of branches. The last column of Table 1 shows energy breakdown of the CGRA for these groups of applications.

In the pie charts of Table 1, the energy consumption of the CGRA is divided into three different portions: PE array (yellow slices), DM module (green slices), and CS process (brown slices). The PE array slice

Table 1
Selected applications as benchmark suite.

Name	Domain	Application list	# of Contexts	CGRA energy breakdown ^a
Livermore Loops	Math.	<i>First Diff</i>	3	
		<i>Hydro</i>	3	
		<i>ICCG</i>	7	
		<i>Inner Product</i>	3	
		<i>Matrix Mul</i>	7	
		<i>State</i>	8	
Array Sorting	Memory	<i>Tri-diagonal</i>	3	
		<i>Bubble sort</i>	5	
		<i>Insertion sort</i>	5	
		<i>Selection sort</i>	7	
		<i>Comb sort</i>	8	
Image Processing	Matrix	<i>Shell sort</i>	6	
		<i>Edge Detection (ED)</i>	8	
		<i>Sharpen</i>	6	
		<i>Gaussian Blur 3×3 (GB)</i>	7	
BDTi Kernels	DSP	<i>24-tap FIR</i>	5	
		<i>IIR</i>	8	
		<i>Vector Addition (Vadd)</i>	2	
		<i>Vector find max (Vmax)</i>	5	
		<i>Vector Dot Product (Vdot)</i>	2	

^aYellow part is for PEs, Green part is for the DM, and Brown part is for the CS process.

shows energy consumption of the PE array in the execution phases of the CGRA. The DM slice shows the energy portion of the data movement between PEs and the DM module in the Execution phase. The CS process portion includes the CM module energy consumption in both idle time and the CS process time, as well as the PE array energy consumption in the CS process. Based on our results, the CS process consumes (on average) 20–53 percentages of the total CGRA energy. The portion of the CS process from the whole energy consumption of CGRA is strongly related to its average number of different contexts. Because, by rising the number of contexts, the number of context switching requests might increase dramatically.

3. Related works

Based on the large design space of CGRAs, there are many options to reduce their energy consumption. Some researchers have used VLSI techniques such as DVFS [30], transistor level reconfigurable cell design [31], or clock and power gating [4,14,32]. Some others have presented novel architectures to improve the energy efficiency of CGRAs for specific applications. These researchers have tried to have a different and deeper look inside their target application group, and have proposed enhanced architectures, such as processor-in-memory (PIM) [2] for MIMD processing applications, multi-grained instruction set extension design [33], multi-level architectures [20], etc.

Some techniques on the mapping process are proposed for reducing energy consumption of the CGRAs. In [34], some mapping-level techniques have been proposed to increase energy efficiency of a NUMA CGRAs. Kim [22] have tried to copy instructions of some PEs to some idle ones to reduce communication cost of the PE array and reduce energy consumption of the CGRAs.

Another option to reduce energy consumption of a CGRA concerns its DM network. Tajammul et al. [2] have proposed a reconfigurable DM network to reduce energy consumption of CGRAs. In [16], data

prefetching is presented to reduce delay of the data transmission in CGRAs. In [5], a hierarchical memory organization is proposed to reduce delay and power of the DM network.

Based on the considerable energy consumption of the CS process (as shown in Table 1), many researchers are working on the CM network and the CS process to improve energy efficiency of CGRAs [15,36,37]. The goal of these techniques is to reduce energy of the CS process through decreasing the number of accesses to the CM module, the CM module size, the CS process delay, the amount of transmitted configuration bits from the CM module, or the number of transition bits in the CM network.

One of the main steps in the CS process is to access the CM module and read new configuration words. Placing more than one configuration register in each PE is a common way to reduce the number of accesses to the CM module [35]. The extra configuration registers play the role of a small hidden cache memory for the CM module with small delay and energy.

To reduce size of the CM module, the number of stored configuration bits have to be decreased. This type of technique is also called context compression technique. First, they try to reduce the amount of required context memory by compressing configuration bits or shrinking their number [8,36,37]. Chung et al. [36] have sorted instructions based on the frequency of their occurrence, and coded the most frequent ones with less bits. In [8], each context is analyzed and special areas with similar instructions inside it are detected. In [37], the CM module size is reduced by reordering instructions in the memory and adding a simple readdressing step before the CM. Using this technique, redundant instructions will be eliminated from the memory and its size will be decreased automatically. By storing only one instruction for each area, the size of the CM module is reduced by up to 50%. Subsequently, both static and dynamic powers of the CM module are decreased, as well as the energy consumption for reading/ writing data from/to it.

Kim and Mahapatra [15] did a statistical analysis of their instruction format. They have found, there is no need to store all fields of an instruction in the configuration cache. Based on the operation of the instruction, some fields can be eliminated in the compression process from that configuration word. They had divided the fields of a configuration word into three different groups: necessary, optional, and unnecessary for PE operations. The necessary fields cannot be eliminated by the compression method, for example 'Opcode' field. However, the existence of optional and unnecessary fields is related to the 'Opcode' and they can be eliminated from the configuration word. Using this technique, they have replaced more than 95% of 32-bit configuration words with 18-bit words. They had stored compressed configurations into a small (18-bit word) CM and used a regular CM for the remaining of the configurations. They could save about 40% power of the CM module.

Wang et al. [5] have presented a lossless dictionary-based configuration compression technique to reduce the CS process cost. Ji and et al. also proposed a dynamic decompression technique to reduce the amount of stored and transferred configurations bits in/from the configuration memory [38]. In [39], another configuration compression technique is proposed to reduce the CS cost, where an extra post-compile step for task remapping to improve the efficiency of the compression method has been used [19].

Another trend to manage the energy efficiency of the CS process is to reduce delay of fetching configuration data from the CM. Two main approaches are proposed for this goal. First, by placing more than one configuration register inside each PE, the number of accesses to the CM module for transmitting new contexts will be decreased [40], and thus the CS process delay will be decreased. Another technique to decrease the number of accesses to CM is to broadcast a fetched configuration word to more than one PE [15] (in case of similarity). The third way to decrease the CS process delay is to prefetch the next configuration words when the PE array is operating in the execution mode [5].

In addition to previous methods, some methods are proposed to reduce the CS process energy consumption by decreasing the number of transmitted configuration bits from the CM to PEs. Rossi et al. [6] extracted frequent patterns from the contexts and assigned them a code. The CM module only sends these codes to the PEs (or clusters), where proportional configuration word will be read from a dictionary. In the last few years, other multiple dictionary-based context compression methods have been proposed [38] with similar ideas. By sending fewer bits than total configuration bits, less dynamic power will be consumed and the CS process energy efficiency will be enhanced.

The last method to mention in this field is to reduce the number of transition bits in the CM network [5]. Authors have used a bus-invert coding [41] on the output of the CM module. Using this technique, they could decrease the energy of the CGRA.

In all aforementioned methods, researchers have looked into the CS process of a CGRA from different points of view. They have proposed their basic techniques, and have also used remapping and memory hierarchy techniques to improve efficiency of their methods. In this paper, we will be also trying to look into the CS process from another point of view. We have focused on the sequence of the fetched contexts from the CM and tried to reduce the number of transitions by unifying the configuration bits.

4. Configuration unification method

In the CS process, configurations have to be transmitted in the CM network and placed on the configuration register of each PE. We are proposing a method, configuration unification method, to reduce the number of transition bits in the CM network and PEs in the CS process. In the configuration unification method, a post-compile program analyzes each configuration word (of the input application) and processes it to reduce transition count of the CS process without changing the functionality of the application.

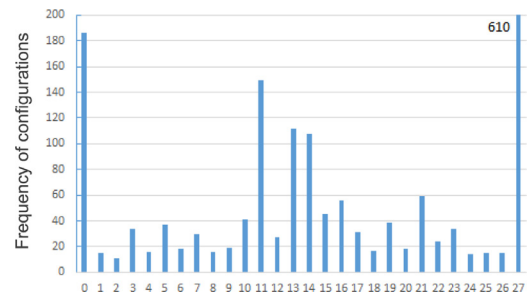


Fig. 3. Frequency of unimportant bits.

In the configuration unification method, fields of each configuration instruction are divided into two categories: important fields and unimportant ones. The important field bits cannot be changed, because they are used to control the PE behavior in its operation. But, unimportant fields are not used for that particular 'Opcode'. For example, in the NOP instruction, all fields except the 'Opcode' are unimportant and it means 27 (out of 32) bits of the NOP instruction can be changed without any effect on the function of the PE. In addition, the immediate number field is unimportant for many instructions, which means 11 (out of 32) bits can be changed. Fig. 3 shows the number of unimportant bits of configuration words of all applications of Table 1. Based on our analysis, in the benchmark group, close to 90% of configuration words have at least one unimportant bit, about 30% have 27 unimportant bits, and only 10% of configuration words did not have any.

To decrease the number of transitions, these unimportant bits have to be replaced with appropriate important ones. In this method, for each application, a library for each PE is made, containing different configuration words of that PE. For example, in Fig. 2, library of PE00 contains only three different instructions, PE03 and PE30 have 2 different instructions, and PE33 have only one instruction. In this method, to find appropriate replacement for unimportant fields, these instructions will be searched. Fortunately, this library can be constructed at compile-time. For example, in PE03, there are two different instructions; one instruction is 'NOP' and the other one is 'broadcast'. As we mentioned before, 27 bits of NOP instruction can be replaced without any changes in the functionality. Hence, the 27 LSBs of the instructions of the contexts #0, #1 and #2, will be replaced with 27 LSBs of the 'broadcast'. Therefore, 27 bits of configuration word of this PE is unified with the same bits and due to the CS process, no transitions will be occurred in them.

On the other hand, our basic CGRA is a token based control flow system and its modules are not activated until they are given adequate tokens. For example, in Fig. 2, PE03 only has one non-NOP instruction and it is only activated if it gets token from its left PE (PE02), and in contexts #0, #1 and #2 no token will be passed from PE02. By using this lead, we can replace all 32 bits of the 'NOP' instruction. By considering this fact, more than 20% of the configuration words of the target applications will be replaced completely.

Another approach to get more unified bits is to design a better ISA and instruction format for the CGRA at the design time. By improving the ISA, the number of unimportant bits and uncompressible configuration words could change, and taller bars would go to the right side of Fig. 3. On the other hand, by affecting more configuration words, more bits can be unified and respectively power consumption might be reduced. Rakossy et al. [34] have used a simple ISA, and thus have affected more than 95% of their configuration words while this factor is 90% for our ISA.

Fig. 4 shows the effect of applying configuration unification method, in terms of reduction of transition counts and the CS power. Using the configuration unification method can reduce the number of transitions by up to 60% (on average 32%) for the applications. On the other hand, by reducing transition bits in the CM network, the CS process power consumption can be decreased by up to 50% (on average

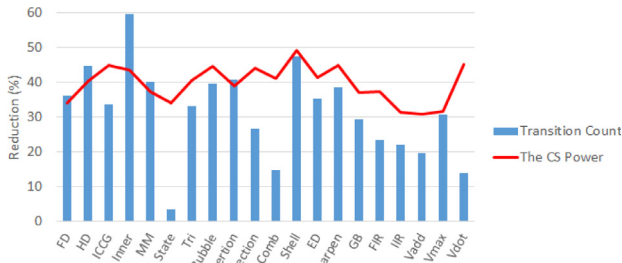


Fig. 4. Efficiency of applying configuration unification method.

40%). The CS power consumption contains both CM network power consumption and PE power consumptions in the CS process. If the configuration word changes rarely, PE transitions occur less, and subsequently the CS power will decrease more.

The configuration unification method can reduce the number of transitions in the CM network due to the CS process. However, it cannot reduce the number of configuration words. In the next section, a compression method will be proposed based on the unified configurations of the contexts.

5. Configuration unification compression

Decreasing the number of transition bits will reduce power of the CS process by affecting the dynamic power of the CGRA. In the modern technologies, the static power cannot be ignored anymore. Hence, in this section we have proposed a new method to reduce the static power of the CGRA as well as its dynamic power.

In a memory unit, almost all memory cells have to be on, waiting for receiving read/write commands, while these active cells consume static power. By increasing the memory size, the amount of the static power grows. In a CGRA, the volume of the context memory is specified by the maximum number of configuration words of its group of applications. In this section, we will propose a compression method to reduce the number of configuration words of each application. The proposed compression method is working along with the unification method, and thus is named configuration unification compression (CUC) method.

In the CUC method, between 27 and 32 configuration bits of more than 30% of the context words can be replaced with other configuration bits. Aside these unimportant bits, some of important bits of two successive configuration words may be the same. By replacing these unimportant bits and having already similar configuration bits, many redundant configuration words will appear in the context memory. Our proposed configuration compression, the CUC method, is based on removing these redundancies.

Applying the CUC method to the target applications will reduce the number of different configuration words of each application. Fig. 5 shows compression ratio ((Uncompressed words – Compress words) / Uncompressed words) obtained by applying the CUC method to different applications. As shown in Fig. 5, for applications of the target group, up to 52% and on average 35% of the configuration words can be removed.

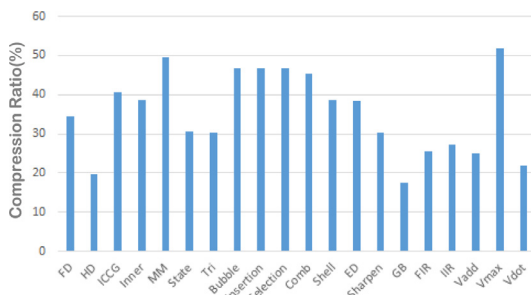


Fig. 5. Compression ratio of the proposed method.

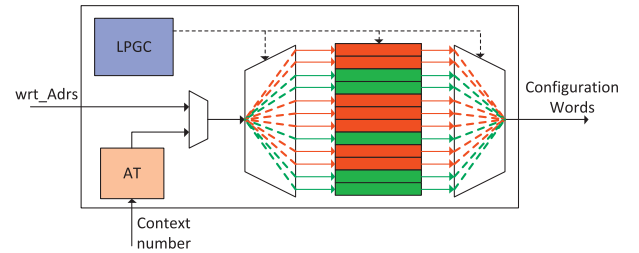


Fig. 6. Modified architecture of the CM.

In the CUC method, to address a removed configuration word, an address translator is added before the CM address decoder.

On the other hand, we have applied the power gating method [4,14,18] on the idle lines of the context memory to decrease static power of the CM. Power gating is a powerful method, although it should be used carefully because of supply voltage fluctuation. When power gating method is applied to a system frequently (low on/off period), some noises will appear in the system [42]. In the CGRA application execution model, context of an application will be transferred to the context memory in the boot time and it will not be changed in the execution phase, hence, using of the power gating technique is safe. Fig. 6 shows the modified architecture of the context memory. In the modified CM module, the 'LPGC' (Line-wise Power Gating Controller) and 'AT' (Address Translator) units are added to the original architecture of the CM unit.

Fig. 7 shows the effect of applying line-wise power gating method (LPG) without and with configuration unification compression method (CUC + LPG) on the power consumption of the CM models. As shown in Fig. 7, applying the LPG method reduces the CM power consumption up to 66% and on average by 28%. If the CUC method is also added to it, the power consumption reduction will be increased by up to 79% and on average 66%.

At the compile-time, some additional metadata will be produced to configure the LPGC and AT modules. They are fetched at the boot-time of each application. Therefore, based on the CUC method, the boot time of each application will take a few cycles more than the regular way. To decrease the number of added clock cycles, the produced metadata will be encoded. The LPGC configuration data is only one bit for every line of the CM, hence, the information of every 32 lines are gathered into one word, and transmitted in one clock cycle. On the other hand, for the AT module, the offset value will not overstep a 4-bit number (for our applications). Hence, the required information for each PE (in our case) will be gathered in one word. Using this technique, the number of added clock cycles will be few and it can be done over the boot time.

As we discussed before, the configuration unification and its compression methods use the advantages of the similarity between configuration words of each PE. However, these two methods cannot increase similarity of the configuration words of application contexts more than their unimportant bits. In the next section, we will describe some useful techniques to increase similarity among the configuration bits of each

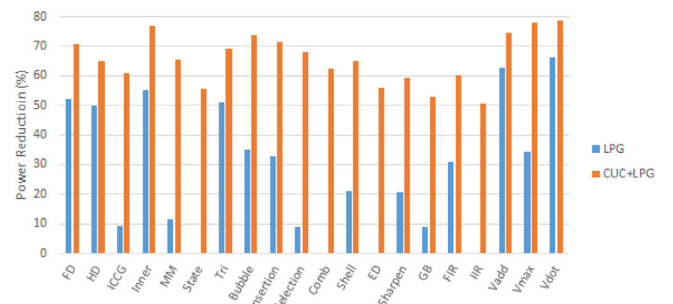


Fig. 7. Power reduction of the CM module.

PE and achieve more power reduction.

6. Increasing similarity of configuration bits

As said in the previous section, some techniques will be described here to increase similarity of the bits of the configuration words of each PE. The proposed techniques have to be applied at different levels from design to mapping. But first, some pre-execution available information about instructions of each PE have to be known and analyzed.

6.1. Pre-execution information about PE instructions

Before starting the execution of an application on a CGRA, some useful information can be extracted from its contexts, which will be used in the configuration unification compression method. These useful points are the number of NOPs per PE and per context, input reordering, estimation of fetching contexts sequence in run-time, and set of different operations in a PE. In the following, by using these useful points, we have suggested some techniques to enhance the efficiency of the proposed CUC method.

6.2. Number of NOPs per PE and context

The NOP is the most typical instruction in contexts of every application. Usually, the ratio of NOP instruction to the whole number of configuration words of an application is in range of 7–67% [43]. That ratio for our applications, is on average about 45%. Compilers try to reduce the number of NOP instruction to increase performance of the CGRA, whereas in this paper, high number of NOPs per PE means more unimportant configuration bits, and thus more unified configuration words. In the best case scenario, from context switching point of view, a PE would have only one non-NOP instruction and does not need to save multiple different instructions in the CM module (such as PE03 and PE30 in Fig. 2). However, this scenario is quite rare.

In many cases (8-out-of-16 PEs of Fig. 2), there is multiple non-NOP instruction. In these cases, we look into the whole context and try to rearranging the NOPs in order to increase the number of unimportant configuration bits in the whole context.

6.3. Input reordering

Input fields are two fields of the configuration word containing 6 bits. In most instructions, these fields cannot be marked as unimportant fields, however, in many cases their order is not important and they can be replaced with each other.

In these cases, the configuration unification method searches all instructions of each PE to find similar first and second inputs. It tries to increase similar bits in the input field by replacing the first and second input bits in those instructions.

6.4. Context fetching prediction

To reduce the CS process energy consumption of a CGRA, the number of unimportant configuration bits of the contexts must be increased. Previously, we supposed the next context is fetched in run-time immediately after the current context. However, in some cases this does not happen. Trying to unify configuration bits of context of a program without knowing the correct behavior of that program may increase energy consumption of the CS process. Based on our analysis, only in 44.3% of the cases the successive context is the same as the next stored context in the CM.

When an application is compiled and mapped to a CGRA, each context is specified with a unique number, CN (context number). After the mapping stage, the sequence of fetching contexts of the application is almost specified. For example, in Fig. 2, contexts #0, #1 and #3 only have one candidate as the next context, and only context #2 has two

candidates (#1 and #3) that can be fetched in run-time. Based on our analysis, in our target applications, more than 80% (on average) of contexts have only one branch instruction. Hence, finding CN of the successive contexts is possible in compile-time.

When the context has only one branch instruction, finding the CN is not hard; whereas in other cases, it is more complicated. When a context has more than one branch, we predict CN of the successive contexts based on the probability of occurrence of each CN. In our ISA, we have only defined two decision operators, ‘=’ and ‘<’. Each decision instruction has one (or more) TRUE (for satisfied condition) and FALSE (for not satisfied condition) ways. When the instruction is ‘=’ its FALSE way and when it is ‘<’, its TRUE way will be considered as the successive CN. By using this method, we could predict the number of the successive context in more than 89.3% of cases.

6.5. Opcode reassignment

In the CU method, the CS process energy consumption can be decreased by reducing the number of transition bits in the configuration words. ‘Opcode’ is an important field of the configuration word, and thus as such, its transitions cannot be reduced by the CU method. Hence, in this part we have proposed another technique to reduce transitions bits of the ‘Opcode’ field in the CS process.

Generally, the architecture of a CGRA is designed based on its target applications. The source code of these applications are available in design time. In addition to the set of CGRA instructions, the set of different instructions of each PE is also clearly specified in design time. By considering this information, the opcode bit assignment of each instruction can be done efficiently. To reduce the number of transition bits in the ‘Opcode’ field, the ‘Opcode’ bit strings with minimum hamming distance should be assigned to the most correlated instructions. The effect of applying this technique to the instructions of the ISA is similar to applying the unification configuration method to the rest of the configuration words.

Fig. 8 shows the effect of applying these four techniques over the CUC method in term of energy reduction of the CM module. In Fig. 8, ‘NOP’ refers to the effect of rearranging NOPs in contexts, ‘Input’ is the effect of input reordering, ‘Sequence’ is the effect of predicting the CN of the successive context, and finally, ‘Opcode’ shows the effect of applying ‘Opcode’ bit reassignment. Based on the results, predicting the CN of the successive contexts technique has greater effect than other techniques.

7. CGRAmulator

To estimate delay and energy consumption of the CGRA, we must implement it in details. We have implemented the mentioned CGRAs in the Section 2.1., at three different levels of abstraction: RT-level model, transistor-level, and instruction set simulation level.

We have described the CGRAs at the RT-level with Verilog language. Then, using the RT-level model we have implemented it at the transistor level in HSPICE, using the 32 nm PTM library [44] for studying the exact behavior of the circuit and its modules. The reported energy consumption of the transistor level is more accurate than RT-level, at

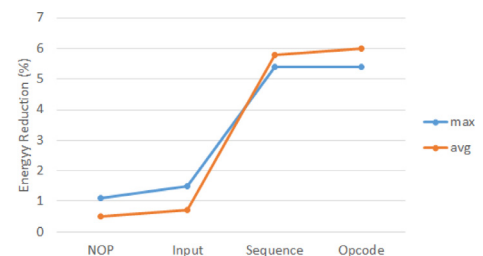


Fig. 8. Energy reduction of the CM module.

the expense of higher simulation time. Also, we have developed a high level cycle accurate instruction set simulator for our CGRAs, in C++, named CGRAMulator. The simulation time at this level is very low, however its reported energy value has to be validated. To verify and validate our simulator, we have run a small version of some little benchmarks at the transistor level model.

CGRAMulator is designed as an instruction-set simulator for CGRAs. CGRAMulator takes contexts of an application and data from user and then runs it on the CGRA. All previous results have been extracted by the CGRAMulator.

7.1. ISA of the CGRAMulator

The CGRAMulator is an instruction-set simulator, which is designed for simulating input contexts of an application on the described CGRA architecture in Section 2.1. ISA of the CGRAMulator is designed based on our input group of applications.

Set of instructions is one of the most important parts of an ISA. Not only it indicates the number and types of the instructions of the CGRA, but also it defines the internal architecture of the PEs. In the designed ISA, there are fourteen instructions in four types: arithmetical/logical ('NOP', 'Brdest', 'Imm', 'Cnt', 'Add', 'Sub', 'Mul', and 'Shift'), comparative ('Equal' and 'LessThan'), memory accessing ('Load' and 'Store'), and context control ('Goto' and 'Exit').

In addition to the set of instructions, ISA also contains the format of instructions. As mentioned in Section 2.1., each instruction has six fields: Opcode, activators and outputs (control fields), input1 and input2, and immediate. The activator bits define number and source of input tokens of the PE that starts executing. The output bits indicate the output port which gets token and results of the operation.

Each PE takes its input values from three different sources: input ports, immediate register, and previously stored output. Each PE has five inout ports from/to the data memory module (M) and its neighbors in the PE array. The internal architecture of the PE is shown in Fig. 9. In each PE, the configuration word is stored in configuration register (CR), output of the ALU can be stored in data register (DR), and immediate part of the instruction will be stored in immediate register (ImmR).

7.2. Application running on the CGRAMulator

Running an application on the CGRAMulator contains multiple phases. In the first phase, input data and contexts have to be transferred into the DM and CM modules. Then, the first configuration has to be fetched to the PEs. Fig. 10 shows the CGRAMulator application running flowchart.

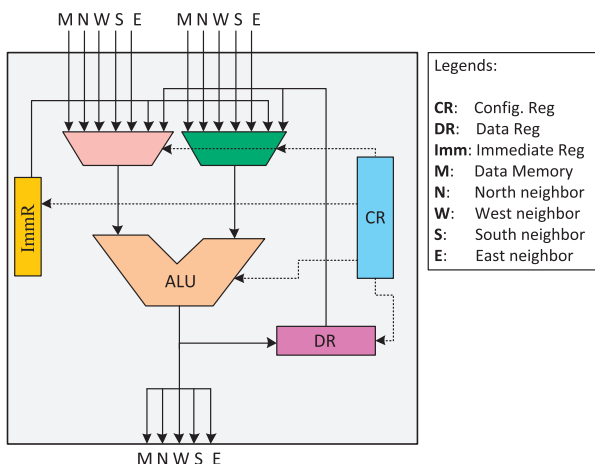


Fig. 9. Internal architecture of a PE.

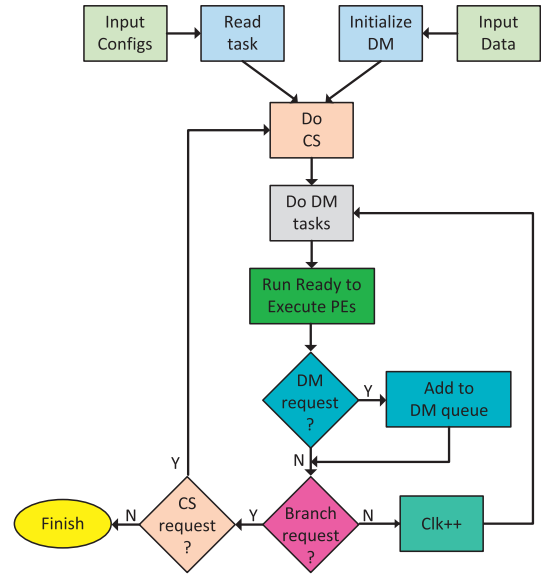


Fig. 10. Application running flowchart on the CGRAMulator.

At the beginning of each clock, DM read/write requests have to be checked and acknowledged. Then, all PEs have to be investigated to find the ready-to-execute ones. In the next step, instructions of the ready-to-execute PEs are executed and their outputs calculated and their DM requests are added to the DM request queue for the next clock cycle.

After executing all ready-to-execute PEs, the 'Branch' requests must be checked. In this step, if a PE requests to execute the CS process, the CGRAMulator will go to the context switching mode. In the CS mode, the CN of the next context goes to the CM and reading from it will be started. But if there is no request for the CS process, the clock will be increased and the control goes to the beginning of the loop.

7.3. Application delay calculation of the CGRAMulator

To calculate execution time of an application execution on the CGRA, multiple factors has to be considered. Execution time of an application can be divided into four parts: writing user's data into the DM, writing configuration words into the CM, delay of the CS process, and context execution. The two first parts can be calculated before starting the execution, whereas the two last parts are related to the contexts of the applications.

Delay of the writing user's data into the DM can be calculated by parsing the input data file of the user. We have named this phase 'DW'. The delay of writing configuration words into the CM module is equal to the number of coded configuration words and generated metadata by the CUC method. We have named this phase of application execution 'CW'.

The delay of the context switching process is related to the CM network model. Before starting the simulation, the user has to specify the desired model of the CM network. Based on the specified model, delay of the CS process will be one clock cycle or more. We have named this phase 'CSD'.

7.4. Energy estimation by the CGRAMulator

Estimating the energy consumed by an application on the CGRA is more complicated than calculating its execution time. There are four main modules in the CGRA architecture: data memory, context memory, control unit and an array of processing elements. In addition to these modules, there are plenty of global wires in the CGRA architecture.

With the technology shrinking of transistors, delay and power of global wires have become more dominant than transistors. Hence, we have considered these global wires in our energy consumption estimation model. To simplify our model, we have modeled global buses by a chain of inverters based on their RC delay [45]. In our CGRA architecture, global wires connect the DM module to PEs of each row of the PE array, and also they connect the CM module to all PEs (based on its CM network model). To estimate delay and energy consumption of a global wire, we have predicted its length, and calculated the number of its equivalent inverters. We have added this inverter chain to the output of that (wire) driving module.

As we mentioned before, a transistor-level model of the CGRA has been implemented in HSPICE. Then, the important parameters have been extracted and inserted to the CGRAMulator. By multiplying the average power consumption of each module by its delay, the amount of its energy can be calculated.

The extracted parameters from the HSPICE model are the static and dynamic energy consumptions of each module. We have called the extracted parameters: PE0, PE1, DM0, CM0, PES, CMS and DMC. The {PE0, DM0, CM0} are the amount of energy that the PE, DM and CM modules consume when they are in idle mode, respectively. PE1 is the amount of energy that a PE consumes on average on instruction execution. The PES and CMS are values of the consumed energy of the PE and CM modules per each transition bit in the CS process, respectively. Finally, DMC is the amount of energy that DM consumes for each DM request from the PE array in each clock cycle.

In the 'DW' phase, the CM and PE modules are idle, but the DM module is in active mode. In the 'CW' phase, the DM and PE modules are idle, but the CM module is activated. In the 'CSD' phase, the DM is idle, but the CM and PEs are in the context switching mode. In this mode, CGRAMulator calculates the transition bits of the CM network and multiplies it to the PES and CMS values to calculate the energy consumption of the CM and PE modules in this phase. In the context execution phase, the CM module is idle, DM and PE modules may be activated or not.

To validate the reported energy consumption by the CGRAMulator, we have simulated some small benchmarks in the HSPICE model and compared their results with the CGRAMulator results. Fig. 11 shows the error of calculated energy consumption by CGRAMulator vs the simulated HSPICE models for the benchmarks ('FD', 'HD', 'Inner', 'Tri' are from Livermore benchmark suite, and 'FS', 'MVM', 'Vadd', and 'Vdot' are vector-based DSP benchmarks). The low level parameters are extracted from two examples and then they have been applied to the CGRAMulator to estimate the energy consumption of these benchmarks. To show the consistency of the CGRAMulator in high number of iterations, the execution phase of each benchmark has been divided into ten equal parts and energy of each part is compared with its HSPICE model.

As shown in Fig. 11, the estimated energy error of the tested benchmarks by the CGRAMulator is less than 2.5%. This error is due to a small difference between the estimated and the real energy consumption of the PE, CM, and DM modules per transition. At the

beginning of the task execution, CGRA passes through two configuration-writing (CW) and data-writing (DW) phases and then it goes into the execution phase. The error of estimated values for writing into the CM and DM modules is small, whereas the error of PE (because of its vast type of operations) is larger. At the beginning of the execution (portion 1 in Fig. 11), the value of the error is small and gradually converges to its final value (based on the frequent types of the operations of the PE).

On the other hand, the simulation time of the CGRAMulator is about 300 ms for the worst case, while for the HSPICE mode, the simulation time varies between 22 h and 34 h.

8. Experimental results

In this section, we will present some features of our experimental setup. To clarify the efficiency of our proposed methods in the different CM network models, the result of applying the proposed methods on them are compared. Then a comparison in terms of area, frequency and energy consumption of our proposed methods and previous works will be presented.

8.1. Experimental setup

We have implemented three individual 4×4 CGRAs to test the efficiency of our proposed methods for all different CM network models. Except the different CM network models, in these CGRAs all other configurations are similar. The common configurations of the CGRAs are a 128-word direct access DM module, a 128-word CM module, and a 2-D mesh topology PE array with 16 PE modules.

Three different CGRA models are CGRA-P, CGRA-L and CGRA-B. The CGRA-P model is using the point-to-point CM network model. In the CGRA-P, the configuration bits of all PEs can be fetched in one cycle. Each PE contains only one configuration register and the CM module is connected to the PE array by a 512-bit wide read port.

The CGRA-L model has an 8-word local direct map configuration cache inside each PE (based on our applications) to store all different configurations of each benchmark and also one configuration register is placed to store the activated configuration word. In this model, there is only one 32-bit read port which is shared among all PEs. To reduce delay of the CS process in this model, all configuration words will be fetched at the boot phase to the local context cache. Hence, delay of the first time configuration fetching is high, however its CS process delay is only one clock cycle.

In the CGRA-B, there is only one 32-bit read port for each column of the CM module, and also there is only one configuration register inside each PE. The area cost of this model is less than previous ones, however its CS process delay is high. In the CGRA-B, each context switching will take 4 clock cycles.

The different CGRA models have been implemented at the transistor level in HSPICE, by using the 32 nm PTM library. To obtain the average power consumption of the CGRAs, some working parameters have to be fixed. The operation frequency of the CGRAs is fixed at 500 MHz. The supply voltage (VDD) of the circuits is set to 0.9 V. Based on our HSPICE results, the number in million transistors of these CGRA models are 1.643, 1.891 and 1.572 for CGRA-P, CGRA-L and CGRA-B models, respectively. CGRA-L has the biggest number of transistor, due to its local context cache. CGRA-P has many global wires, modeled by inverter chains, thus its transistor count is more than CGRA-B.

8.2. Area overhead

In this paper, we have proposed two methods to reduce the CS process energy consumption. In the first method, the configuration unification has no effect on the architecture of the CGRA. In the second proposed method, the configuration unification compression method requires some changes in the architecture of the CM module. The area

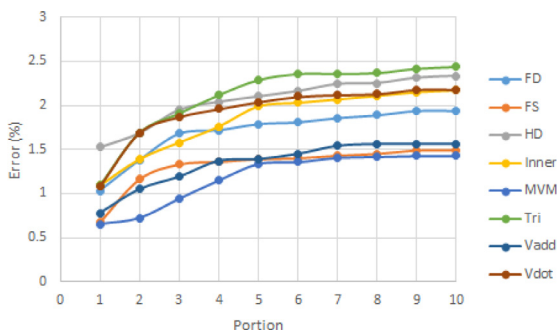


Fig. 11. Generated error of the CGRAMulator.

Table 2
Comparison to the base architecture.

	AO (%)	DD (cycles)	CR (%)	CM (%)	CGRA (%)
[15]	10.35	0	38.67	25.13	12.11
[35]	80	–	40–56	16.12	12.18
[33]	–	–	57	20.78	10.35
[36]	–	–	51	37.07	15.46
[8]	–	–	84.82	18.09	9.74
[37]	2–3	–	28.56	39.67	9.94
[38]	6	3	72	17.94	17.26
CU	0	0	0	49.39	5.63
CUC	10.51	0	44.56	81.67	19.34

overhead (increasing in terms of number of transistors) due to the application of this method on the CM module is about 10.5%, while it is less than 2% for the CGRA. The area overhead of our proposed methods is compared to previous works in Table 2.

8.3. Decompression delay overhead

In addition to the area overhead, there is another cost of using compression methods, which is the decompression delay. Usually, the context compressing process is done in compile-time. However, the decompression process has to be done in run-time, inside of each PE. In the CUC method, there is no need to decompress the encoded configuration words in the PE. Hence, no decompression delay is imposed to the CS process, using this method. Table 2 shows the decompression delay of previous works.

8.4. Energy consumption reduction

The main purpose of our work is to reduce the CGRA energy consumption, while it is running an application. In this paper, we have proposed two methods to obtain this goal. The first method only decreases the power consumption of the CGRA; whereas the second one, i.e. the compression method, affects both power consumption and delay of execution. Hence, in this paper, we have selected the energy consumption of the CGRA as the main efficiency factor of the methods.

We have shown the effect of our proposed methods on the CM energy consumption in the previous sections, individually. In the previous results, only the effect of the methods on the CGRA-P model is reported. The effect of the proposed methods on all discussed models of CGRAs will be also presented. Fig. 12 shows the results of applying the CUC method on the energy consumption of the CM module and the CGRA with different model of CM networks. The results of Fig. 12 show the average effect of applying the proposed method on the applications of each group.

Based on our results, the application of the CU method can reduce up to 50% (on average 36%) the CM energy and up to 14% (on average 5%) the CGRA energy. In the CGRA-P model, the number of interconnects and their capacitances of the CM network are higher than other models; hence, each transition bit consumes more energy in this model. Therefore, the effect of applying this method on the CGRA-P is greater than the others.

Based on our results, using the original CUC method (CUC-O) can reduce up to 80% (on average 71%) the CM energy and up to 22.5% (on average 13%) the CGRA energy consumption. Contrary to the CU method, the efficiency of this method relies on the reduction of the CM module static power. Hence, efficiency of applying this method on the CGRA-B is more than other models.

Finally, we have used four techniques to increase the efficiency of the original CUC method (CUC-E). Efficiency of the CUC-E method is higher than the original one (CUC-O). Based on the results, applying the enhanced version of the CUC method (CUC-E) on the contexts can reduce up to 84% (on average 81%) of the CM energy and up to 24% (on

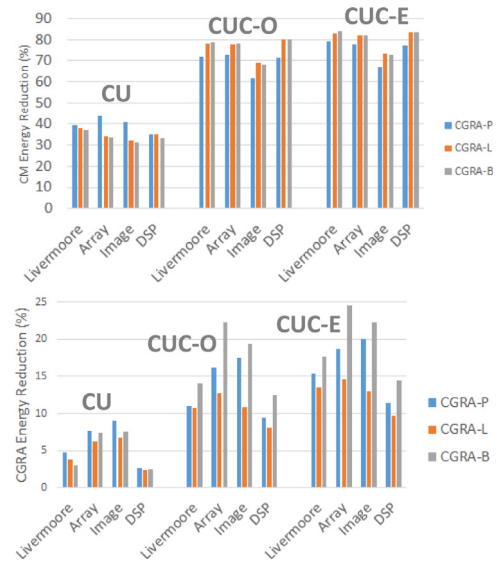


Fig. 12. Effect of applying the proposed method on the energy of the CM module and CGRA with different CM network models.

average 19%) of the CGRA energy consumption. Similar to the CUC method, the best results have been obtained by applying the method on the CGRA-B architecture.

8.5. Compare with previous works

The efficiency of our proposed methods has been investigated previously. However, their advantages and disadvantages over previous works have not been discussed. Table 2 compares the efficiency of our proposed methods (CU and CUC) and some previous works to the base architecture in terms of area overhead (AO), the CM module energy reduction (CM), CGRA energy reduction (CGRA), compression ratio (CR), and finally run-time decompression delay (DD). We have modeled several methods [8,33–38] and added them to the CGRAMulator to evaluate the above parameters. We have applied these methods on all CM networks (P-, B-, and L- model) and reported only the best value in Table 2.

As shown in Table 2, not only our proposed methods can achieve more energy reduction from both CM module and CGRA, but also their area overhead and decompression delay are less than previous works.

9. Conclusion

In this paper we presented two novel methods to decrease energy consumption of CGRAs. First, the configuration unification method reduces the number of transitions due to the context switching process of a CGRA. As the second method, a context compression method is presented to reduce the number of stored configuration words in the context memory of the CGRA. Moreover, an online instruction-set simulator tool was presented in this paper, which can execute input applications and calculate their delay and energy consumption. We have validated its results with HSPICE model of the CGRA. We described different CM network models of the CGRAs and investigated the application of our proposed methods efficiency. Based on our results, up to 83.7% (24.6%) and on average 81.6% (19.3%) of the CM (CGRA) energy can be reduced by using our proposed methods, respectively.

Acknowledgment

This research was in part supported by a grant from the Institute for Research in Fundamental Sciences (IPM) (No. CS1397-4-140).

References

- [1] G. Kuzmanov, The MOLEN polymorphic media processor, *IEEE Trans. Comput.* 53 (11) (2004) 199.
- [2] M.A. Tajammul, M.A. Shami, A. Hemani, S. Moorthi, NoC based distributed partitionable memory system for a coarse grain reconfigurable architecture, *Proceedings of the IEEE International Conference on VLSI Design*, 2011, pp. 232–237.
- [3] L. Liu, et al., An energy-efficient coarse-grained reconfigurable processing unit for multiple-standard video decoding, *IEEE Trans. Multimedia* 17 (10) (2015) 1706–1720.
- [4] C. Kim, M. Chung, Y. Cho, M. Konijnenburg, S. Ryu, J. Kim, ULP-SRP: ultra low-power samsung reconfigurable processor for biomedical applications, *ACM Trans. Reconfigurable Technol. Syst.* 7 (3) (2012) 1–15.
- [5] Y. Wang, et al., On-chip memory hierarchy in one coarse-grained reconfigurable architecture to compress memory space and to reduce reconfiguration time and data-reference time, *IEEE Trans. Very Large Scale Integr. Syst.* 22 (5) (2014) 983–994.
- [6] D. Rossi, F. Campi, S. Spolzino, S. Pucillo, R. Guerrieri, A heterogeneous digital signal processor for dynamically reconfigurable computing, *IEEE J. Solid State Circuits* 45 (8) (2010) 1615–1626.
- [7] G. Dimitroulakis, M.D. Galanis, C.E. Goutis, Exploiting the distributed foreground memory in coarse grain reconfigurable arrays for reducing the memory bottleneck in DSP applications, *Proceedings of the 5th WSEAS Int. Conf. on Signal, Speech and Image Processing*, 2005, pp. 132–137.
- [8] B. Liu, W.-Y. Zhu, Y. Liu, P. Cao, A configuration compression approach for coarse-grain reconfigurable architecture for radar signal processing, *2014 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery*, 2014, pp. 448–453.
- [9] R. Bott, ABRES: an architecture with tightly coupled vliw processor and coarse-grained reconfigurable matrix, *FPL* 03, 2003, pp. 61–70.
- [10] C. Plessl, M. Platzner, Zippy - a coarse-grained reconfigurable array with support for hardware virtualization, *2005 IEEE International Conference on Application-Specific Systems, Architecture Processors (ASAP'05)*, 2005, pp. 213–218.
- [11] H. Singh, M.H. Lee, G. Lu, F.J. Kurdahi, N. Bagherzadeh, E.M. Chaves Filho, MorphoSys: an integrated reconfigurable system for data-parallel and computation-intensive applications, *IEEE Trans. Comput.* 49 (5) (2000) 465–481.
- [12] A. Chattopadhyay, Ingredients of adaptability: a survey of reconfigurable processors, *VLSI Des* 2013 (2013).
- [13] S. Khawam, I. Nousias, M. Milward, Y. Yi, M. Muir, T. Arslan, The reconfigurable instruction cell array, *IEEE Trans. Very Large Scale Integr. Syst.* 16 (1) (2008) 75–85.
- [14] M. Shafique, L. Bauer, J. Henkel, Adaptive energy management for dynamically reconfigurable processors, *IEEE Trans. Comput. Des. Integr. Circuits Syst.* 33 (1) (2014) 50–63.
- [15] Y. Kim, R.N. Mahapatra, Dynamic context compression for low-power coarse-grained reconfigurable architecture, *IEEE Trans. Very Large Scale Integr. Syst.* 18 (1) (2010) 15–28.
- [16] L. Chen, J. Tarango, T. Mitra, P. Brisk, A just-in-time customizable processor, *IEEE/ACM International Conference on Computer-Aided Design, Digest of Technical Papers, ICCAD*, 2013, pp. 524–531.
- [17] M.B. Taylor, et al., The raw microprocessor: a computational fabric for software circuits and general-purpose programs, *IEEE Micro*. 22 (2) (2002) 25–35.
- [18] J. Cong, H. Huang, C. Ma, B. Xiao, P. Zhou, A fully pipelined and dynamically composable architecture of cgra, *Proceedings of 22nd International Symposium on Field-Programmable Custom Computing Machines, FCCM* 2014, 2014, pp. 9–16.
- [19] S.M.A.H. Jafri, M. Daneshdalan, N. Abbas, G.S. Leon, A. Hemani, TransMap: transformation based remapping and parallelism for high utilization and energy efficiency in CGRAs, *IEEE Trans. Comput.* 65 (11) (2016) 3456–3469.
- [20] S. Swanson, A. Schwerin, M. Oskin, WaveScalar, *International Symposium on Microarchitecture*, 2003, p. 291.
- [21] A. Ashammagari, Exploiting STT-NV technology for reconfigurable, high performance, low power, and low temperature functional unit design, *Proceedings of the conference on Design, Automation & Test in Europe. European Design and Automation Association*, 2014, p. 335.
- [22] Y. Kim, Power-efficient configuration cache structure for coarse-grained reconfigurable architecture, *J. Circuits, Syst. Comput.* 22 (3) (2013) 1350001.
- [23] G. Ansaloni, P. Bonzini, L. Pozzi, EGRA: A coarse grained reconfigurable architectural template, *IEEE Trans. Very Large Scale Integr. Syst.* 19 (6) (2011) 1062–1074.
- [24] H. Park, Y. Park, S. Mahlke, Reducing control power in CGRAS with token flow, *Proceedings of Workshop on Optimizations for DSP and Embedded Systems*, 2009.
- [25] H. Amano, M. Kimura, N. Ozaki, Removing context memory from a multi-context dynamically reconfigurable processor, *Proceedings - IEEE 6th International Symposium on Embedded Multicore SoCs, MCSoc* 2012, 2012, pp. 92–99.
- [26] Livermore loops benchmark. [Online]. Available: <http://www.netlib.org/benchmark/livermore>. [Accessed: 29-Jun-2017].
- [27] Sorting algorithms | Joshua Kehn. [online]. Available: <http://www.joshuakehn.com/2010/10/1/SortingAlgorithms.html>. [Accessed: 11-Dec-2017].
- [28] Kernel (image processing) [online]. Available: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)). [Accessed: 18-Dec-2017].
- [29] BDTI DSP Kernel Benchmarks™ | Berkeley Design Technology Inc. [online]. Available: <https://www.bdti.com/Services/Benchmarks/DKB>. [Accessed: 18-Dec-2017].
- [30] S. Copen, R. Reed, PipeRench: a reconfigurable architecture and compiler, *Computer* 33 (4) (2000) 70–77.
- [31] R. Hartenstein, A decade of reconfigurable computing: a visionary retrospective, *Design, Automation and Test in Europe, DATE*, 2001, pp. 642–649.
- [32] E. Mirsky, A. DeHon, Matrix: a reconfigurable computing architecture with configurable instruction distribution and deployable resources BT, *Proceedings of the 1996 IEEE Symposium on FPGAs for Custom Computing Machines*, 1996, pp. 157–166.
- [33] S. Yin, C. Yin, L. Liu, M. Zhu, Y. Wang, S. Wei, Reducing configuration contexts for coarse-grained reconfigurable architecture, *ISCAS 2012 - 2012 IEEE International Symposium on Circuits and Systems*, 2012, pp. 121–124.
- [34] Z.E. Rakossy, D. Stengele, A. Acosta-aponte, S. Chafekar, P. Bientinesi, anupam chattopadhyay, Scalable and efficient linear algebra kernel mapping for low energy consumption on the layers CGRA, *ARC* 2015, 2015, pp. 301–310.
- [35] S. Park, K. Choi, An approach to code compression for CGRA, *Proceedings of the 3rd Asia Symposium on Quality Electronic Design, ASQED* 2011, 2011, pp. 240–245.
- [36] M.K. Chung, Y.G. Cho, S. Ryu, Efficient code compression for coarse grained reconfigurable architectures, *Proceedings - IEEE International Conference on Computer Design: VLSI in Computers and Processors*, 2012, pp. 488–489.
- [37] B. Egger, et al., A space- and energy-efficient code compression / decompression technique for coarse-grained reconfigurable architectures, *Proceedings of the 2017 International Symposium on Code Generation and Optimization. IEEE Press*, 2017, pp. 197–207.
- [38] C. Ji, D. Zhang, Y. Gong, B. Liu, Optimization of coarse-grained reconfigurable processor based on dynamic decompression of configuration contexts, *Proceedings of 2015 4th International Conference on Computer Science and Network Technology, ICCSNT* 2015, 2015, pp. 808–811.
- [39] S.M.A.H. Jafri, et al., Morphable compression architecture for efficient configuration in CGRAS, *Proceedings - 2014 17th Euromicro Conference on Digital System Design, DSD* 2014, 2014, pp. 42–49.
- [40] R. Koenig, L. Bauer, T. Stripf, M. Shafique, W. Ahmed, J. Becker, KAHRISMA: a novel hypermorphic reconfigurable-instruction-set-multi-grained-array architecture, *Design, Automation & Test in Europe Conference*, 2010, pp. 819–824.
- [41] M.Y. Qadri, H.S. Gujarathi, K.D. McDonald-Maier, Low power processor architectures and contemporary techniques for power optimization - A review, *J. Comput.* 4 (10) (2009) 927–942.
- [42] S. Kim, S.V. Kosonocky, D.R. Knebel, K. Stawiasz, D. Heidel, M. Immediato, Minimizing inductive noise in system-on-a-chip with multiple power gating structures, *European Solid-State Circuits Conference*, 2003, pp. 635–638.
- [43] Y. Kim, R.N. Mahapatra, Dynamic context management for low power coarse-grained reconfigurable architecture, *Proceedings of the 19th ACM Great Lakes symposium on VLSI - GLSVLSI '09*, 2009, p. 33.
- [44] Predictive Technology Model. [Online]. Available: http://ptm.asu.edu/modelcard/LP/32nm_LP.pm. [Accessed: 03-Jul-2017].
- [45] C. Hernández, F. Silla, J. Duato, A Methodology for the characterization of process variation in NoC links, *Date* 2010, 2010, pp. 685–690.



Mohammad Hossein Sargolzaei received his B.Sc. and M.Sc. Degree in computer engineering from Shahid Bahonar University and University of Tehran in 2006 and 2009, respectively. He is a lecturer at the University of Sistan and Baluchestan. He is currently a Ph.D. student at the University of Tehran. His research interests include reconfigurable system design, and low power system design.



Siamak Mohammadi received his B.Sc., M.Sc. and Ph.D. degrees from the University of Paris Sud Orsay, France in 1990, 1992 and 1996, respectively, all in electrical engineering. During his Ph.D. he was supported by a grant from the Ministry of Education of France. From 1997 to 1999 he was a Research Associate with the Department of Computer Science, University of Manchester, England. In 1999 he moved to Canada and worked at Cogency Semiconductor Inc. in Toronto until 2003 and then at ATI Technologies Inc. until 2005. Currently he is an Associate Professor in School of Electrical and Computer engineering, at the University of Tehran, Iran. He has 20 years' experience in low power VLSI design, verification, asynchronous design, and on-chip interconnects in GALS NoCs. He contributed to the design of the Amulet asynchronous ARM microprocessor.